

# Medical RAG Knowledge Base — 部署指南

版本: v1.1 | 更新日期: 2026-05-15

适用环境: Windows/Linux + Docker (Ollama + Python 3.12) | 可选: GPU + conda myenv

核心交付: CLI + REST API + Docker Compose 三接口统一部署

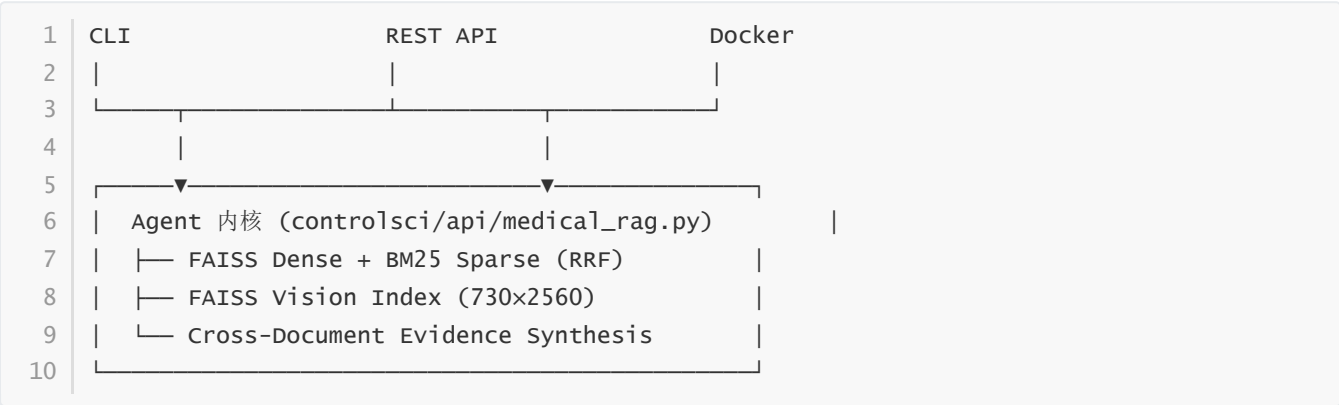
## 1. 系统概述

本系统基于 MinerU 文档解析引擎构建面向医疗文献的高质量 RAG 知识库，支持从 PMC 文献 PDF 到语义检索的端到端自动化处理。

### 1.1 核心能力

| 能力         | 说明                                                                 |
|------------|--------------------------------------------------------------------|
| 医疗文献解析     | MinerU 批量解析 PMC 开放获取文献，保留完整 IMRAD 章节结构                             |
| 语义级切片      | 层级语义树而非扁平正则，包含 Primary outcome / Subgroup / Sensitivity 等 20+ 医学标签 |
| Hybrid 检索  | FAISS Dense (qwen3-embedding:4b, 2560-dim) + BM25 Sparse, RRF 融合   |
| QLoRA 领域微调 | 4B adapter, ChatML PPL -47.0%, Re-ranker MRR +0.044                |
| 视觉注入       | MiMo-V2.5 为 730 张医学图表生成描述 → FAISS 视觉索引 → 双通道 RRF                   |
| 跨文献证据合成    | 多篇文献检索 → 一致性判断 → QA 格式输出 (对齐 MinerU 官方示例)                          |
| 14 源评分矩阵   | 8 API + 6 本地 Judge, 1,050 条评分记录                                    |

### 1.2 三接口架构



## 2. 源码仓库 Docker live mode

独立提交包默认用于 evidence-only 复核; Docker live mode 需要在完整源码仓库根目录运行，不能把 standalone final bundle 理解为自包含 Docker API 包。

## 2.0 构建边界

Track 3 提交包区分两种运行模式：

| 模式                            | 入口                                                                                                     | 适用场景                         | 边界                                                                      |
|-------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------|-------------------------------------------------------------------------|
| final bundle<br>evidence-only | <code>run\run_task3_rag_flywheel.ps1 -EvidenceOnly</code>                                              | 评审独立提交包中的证据 JSON             | 不启动 API，不构建 Docker，不要求 <code>benchmark/controlsci/starboard</code> 源码目录 |
| source repo<br>live mode      | <code>docker compose -f _final_submission_by_track/track3_medical_rag/run/docker-compose.yml up</code> | 在完整源码仓库中启动 live API / Docker | 需要源码根目录包含 <code>benchmark/</code> 、 <code>controlsci/</code> 和索引数据      |

Docker smoke 属于环境依赖检查；如果未实际运行 Docker daemon、端口服务和容器健康检查，只能记录为 `not_run_environment_dependency`，不得写成 `passed`。

## 2.1 前置条件

- Docker Desktop
- 可用磁盘空间 > 5 GB（Ollama 镜像 + qwen3-embedding:4b 模型 + 索引文件）
- 无需宿主机 Python / Conda / GPU；但需要完整源码仓库和 `data/sources_medical` 索引数据

## 2.2 启动

```
1 # 完整源码仓库根目录
2 docker compose -f _final_submission_by_track/track3_medical_rag/run/docker-compose.yml up
```

首次启动流程：

1. Ollama 容器自动拉取 `qwen3-embedding:4b` 模型
2. FAISS 索引自动加载（34 MB text + 7 MB vision）
3. REST API 在 `http://localhost:8001` 就绪

端口说明：`8001` 是独立 Docker/CLI 部署示例端口；本地 Starboard 工作台默认连接 `17001`。standalone final bundle 可用 `un\verify_task3_demo.ps1 -EvidenceOnly` 复核已提交证据，完整源码仓库可通过统一 reviewer demo 复核 live mode。

## 2.3 验证

```
1 # 语义检索
2 curl "http://localhost:8001/api/evidence/search?q=primary+endpoint"
3
4 # 视觉检索（含图片描述）
5 curl "http://localhost:8001/api/evidence/search?q=Kaplan-
Meier+survival&vision=true&k=3"
6
7 # 跨文献证据合成
8 curl -X POST http://localhost:8001/api/evidence/synthesize \
9     -H "Content-Type: application/json" \
10    -d '{"query": "胰岛素泵闭环控制效果", "k": 5}'
11
12 # 健康检查
13 curl http://localhost:8001/api/health
```

## 2.4 REST API 端点

| 端点                             | 方法   | 功能                                                        |
|--------------------------------|------|-----------------------------------------------------------|
| /api/evidence/search           | GET  | 语义检索，返回 top-k chunk (Hybrid RRF) ; ?vision=true 启用双通道视觉融合 |
| /api/evidence/synthesize       | POST | 跨文献证据合成，返回 QA 格式报告                                        |
| /api/evidence/report/{task_id} | GET  | 查询历史合成报告                                                  |
| /api/health                    | GET  | 健康检查（含 FAISS 文本/视觉索引状态）                                   |

# 3. CLI 部署（需要 Conda + Python）

## 3.1 环境准备

```
1 # Conda 环境（Python 3.12）
2 conda create -n myenv python=3.12
3 conda run --no-capture-output -n myenv pip install fastapi uvicorn httpx numpy faiss-cpu
rank-bm25 scikit-learn scipy matplotlib
4
5 # Ollama 本地服务（用于 qwen3-embedding:4b）
6 ollama pull qwen3-embedding:4b
```

## 3.2 单步执行

```
1 # 设置编码
2 $env:PYTHONIOENCODING='utf-8'
3
4 # 1. 医疗文献切片（medical_mode）
```

```
5 # windows + Conda 环境下不要使用多行 python -c; 请先在 _scratch\build_medical_chunks.py 写入
  以下逻辑后运行:
6 # from pipeline.chunk_corpus import build_chunks
7 # build_chunks('data/sources_medical/md', medical_mode=True,
  chunks_dir='data/sources_medical/chunks')
8 conda run --no-capture-output -n myenv python _scratch\build_medical_chunks.py
9
10 # 2. 构建索引 (文本 + 视觉, 需先完成 MinerU 解析)
11 conda run --no-capture-output -n myenv python -m controlsci.medical.indexing
12
13 # 3. 启动 API
14 conda run --no-capture-output -n myenv uvicorn controlsci.api.medical_rag:app --host
  127.0.0.1 --port 8001
```

本地 Starboard 工作台同机联调时可改用:

```
1 conda run --no-capture-output -n myenv python -m controlsci.api.medical_rag --host
  127.0.0.1 --port 17001
```

## 4. 一键复现 (全流程)

```
1 # standalone final bundle evidence-only 复核
2 .\run\run_task3_rag_flywheel.ps1 -EvidenceOnly
3
4 # 完整源码仓库中的 PMC 下载到 RAG 检索全流程入口
5 run_medical_agent.ps1
```

run\_medical\_agent.ps1 是完整源码仓库入口, 执行内容:

- 1. PMC E-utilities API 检索 → 下载 100 篇文献
- 2. MinerU 批量解析 → 结构化 .md
- 3. 医疗章节本体切片 → 3,348 chunks
- 4. FAISS + BM25 索引构建
- 5. 视觉描述生成 → FAISS 视觉索引
- 6. REST API 启动

## 5. 硬件要求

| 部署方式           | 最低配置              | 推荐配置               |
|----------------|-------------------|--------------------|
| Docker         | 4 核 CPU, 8 GB RAM | 8 核 CPU, 16 GB RAM |
| CLI (CPU-only) | 4 核 CPU, 8 GB RAM | 同上                 |
| CLI (GPU)      | RTX 2060 6GB      | RTX 5090 24GB      |

GPU 可选: 仅用于 Ollama 本地推理加速。Docker 版使用 Ollama 容器内推理, 自动利用可用 GPU。

## 6. 故障排查

| 问题                    | 症状                                                           | 解决                                                                              |
|-----------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------|
| Docker 启动后 8001 端口无响应 | <code>curl</code> 返回 Connection refused                      | 等待 ollama pull 完成（首次约 2-5 min），查看 <code>docker compose logs</code>              |
| FAISS 索引未加载           | <code>/api/health</code> 返回 <code>index_loaded: false</code> | 检查 <code>data/sources_medical/index/</code> 下是否存在 <code>medical.index</code>    |
| Ollama 模型未拉取          | <code>embedding</code> 返回空                                   | <code>docker compose logs ollama</code> 确认 pull 完成                              |
| 端口 8001 被占用           | <code>Bind for 0.0.0.0:8001 failed</code>                    | 释放端口或修改 <code>docker-compose.yml</code> ports 映射                                |
| 本地工作台无法连接             | <code>localhost:3000</code> 显示 API 离线                        | 确认 <code>controlsci.api.medical_rag</code> 已在 <code>17001</code> 启动，或同步修改前端代理配置 |
| MinerU 解析失败           | chunk 数为 0                                                   | 检查 MinerU API 是否在 <code>http://127.0.0.1:8000</code> 就绪                         |